解决方案。 把这些大数组移出栈，放进 FAT32 私有的静态工作区 `fat_work`。所有 FAT32 操作本来就持 `fat32_lock` 串行执行，一个工作区就够，不必加大每个进程的内核栈。反汇编确认栈帧降到 `fat32createfile` 464 字节、`fat_find_path` 176 字节。create 路径上加了更细的阶段日志（`create lookup`、`create reserve`、`extending root directory`），可以直接区分是查找、保留目录项还是分配簇出了问题；之前对分配器的优化也保留下来。修复后的完整日志见 7.9.10 小节。

小结。 回头看这十一个问题，有效的做法是一样的：让每一层都有自己的日志，日志里带上能判读的原始数值（`flags`、`fsel`、`levels`、`status`、`esr`、`exists`），然后只把日志实际证明了的东西当作结论。几次走弯路，都是因为把“两行日志挨在一起”当成了因果（问题 8），或者先修了猜测中的原因而没有确认程序是否真的走到那里（问题 11）。

7.9.12 连续取流与 HTTP 推流

7.9.10 小节的日志里，每次 `read()` 都完整地上电、预热、拍一帧、断电，一帧要一秒左右，这样只能拍照，做不了视频。现在的驱动把“上电并开始出流”和“抓一帧”拆开了，在此基础上，用户程序 `camserver` 把画面以 MJPEG 的形式通过 HTTP 推给浏览器。

内核：传感器一直出流。 `camread()` 第一次被调用时执行 `cam_stream_start()`：上电、启动 Unicam 和传感器的 stream、等预热结束，这时 Unicam 停在零长度的 dummy 缓冲上（状态 STREAMING）。第一次取帧时排上第一个槽，此后两个 DMA 槽轮流接收，读者从完成队列里取帧（7.9.8 小节）。传感器始终不停，所以从第二帧起不再有上电和预热的开销。只有 `/dev/video0` 的最后一个引用被关闭时，`camrelease()` 才调用 `cam_stream_stop()` 停流断电。

这带来一条使用约定：连续读帧的程序要一直开着同一个文件描述符。两个进程各自打开、读一帧、关闭，会让传感器反复重启。

用户态：一个进程扇出给所有观看者。 图 7.36 是 `camserver` 第一版的结构：抓帧、编码、发送在同一个循环里串行进行。后来它被拆成了三级流水线，见 7.9.13 小节。

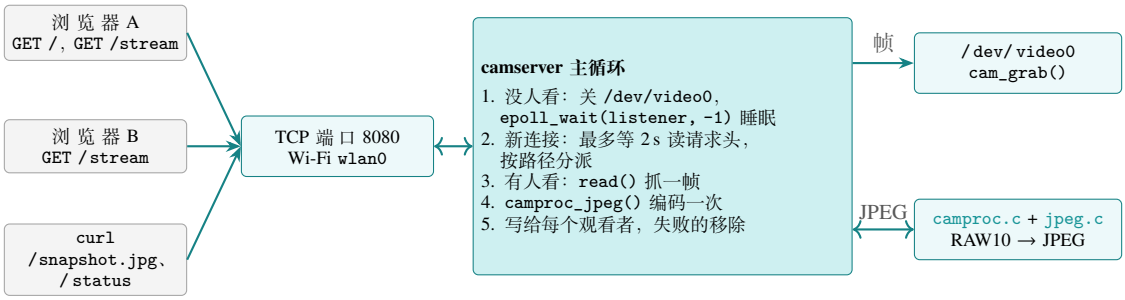


图 7.36: `camserver` 的结构。一次采集、一次编码，写给全部观看者；没人观看时传感器断电

之所以不用“每个连接 fork 一个子进程”的常见写法，有两个原因。第一，xv6 的 TCP 连接归调用 `accept` 的进程所有（内核检查 `c->owner == myproc()->pid`），不能交给子进程。第二，`/dev/video0` 只有一路传感器流，多个进程分别打开会互相停流重启。所以 `camserver` 是单进程的：每抓一帧只编码一次，再依次写给所有观看者；N 个观看者只花一份采集和编码的代价。缺点是一个很慢的客户端会拖慢所有人：它的 TCP 发送队列满了，`write()` 就会阻塞。

新连接在帧与帧之间处理。主循环用一个只监视监听 socket 的 epoll：没人观看时 `epoll_wait(-1)` 睡眠；有人观看时超时为 0，只是看一眼有没有新连接。浏览器经常先建立“预连接”而不发请求，如果直接 `read()` 请求头，整个推流就会被卡住。因此读请求头之前，先用另一个 epoll 最多等 2 秒，等不到就关闭连接。